

# AFRILANG Stdlib

## std/c/parseexpr

**Français.** Bibliothèque AFRILANG 'std/c/parseexpr' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : exprLen, exprUpper, exprLower, exprTrim, exprSplit, exprJoin, exprReplace.

```
'import "std/c/parseexpr"' · 'use parseexpr'
```

```
- 'exprLen(s text) → number' - 'exprUpper(s text) → text' - 'exprLower(s text) → text' - 'exprTrim(s text) → text' - 'exprSplit(s text, delim text) → list text' - 'exprJoin(parts list text, delim text) → text' - 'exprReplace(s text, from text, to text) → text'
```

**English.** AFRILANG library 'std/c/parse-expr' (tier complex, category complex). Exports 7 function(s): exprLen, exprUpper, exprLower, exprTrim, exprSplit, exprJoin, exprReplace.

```
'import "std/c/parseexpr"' · 'use parseexpr'
```

```
- 'exprLen(s text) → number' - 'exprUpper(s text) → text' - 'exprLower(s text) → text' - 'exprTrim(s text) → text' - 'exprSplit(s text, delim text) → list text' - 'exprJoin(parts list text, delim text) → text' - 'exprReplace(s text, from text, to text) → text'
```

|                       |                      |
|-----------------------|----------------------|
| Catégorie / Category  | Modules complex (c/) |
| Niveau / Tier         | complex              |
| Fonctions / Functions | 7                    |

June 16, 2026

## Contents

## Français

### 1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/parseexpr' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : `exprLen`, `exprUpper`, `exprLower`, `exprTrim`, `exprSplit`, `exprJoin`, `exprReplace`.

'import "std/c/parseexpr"' · 'use parseexpr'

```
- `exprLen(s text) → number`
- `exprUpper(s text) → text`
- `exprLower(s text) → text`
- `exprTrim(s text) → text`
- `exprSplit(s text, delim text) → list text`
- `exprJoin(parts list text, delim text) → text`
- `exprReplace(s text, from text, to text) → text`
```

### 2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/parseexpr"
use parseexpr
```

Niveau : complex · Catégorie : Modules complex (c/)  
Domaines avancés – graphes, NLP, réseaux complexes.

### 3. Référence API

- `exprLen(s text) → number`•  
`exprUpper(s text) → text`•  
`exprLower(s text) → text`•  
`exprTrim(s text) → text`•  
`exprSplit(s text, delim text) → list`•  
`exprJoin(parts list text, delim text) → text`•  
`exprReplace(s text, from text, to text) → text`

### 4. Exemples d'utilisation

```
import "std/c/parseexpr"
use parseexpr
```

```
create result = exprLen(/* args */)
say result
```

```
create result = exprUpper(/* args */)
say result
```

```
create result = exprLower(/* args */)
say result
```

```
create result = exprTrim(/* args */)
say result
```

```
create result = exprSplit(/* args */)
say result
```

```
create result = exprJoin(/* args */)
say result
```

## 5. Bonnes pratiques

- Préférez ``import "std/c/parseexpr"`` en tête de fichier.
- Lancez ``afri-lang check`` avant de livrer.
- Combinez avec les modules `stdlib` de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

## 6. Annexe — source

module `parseexpr`

```
function exprLen(s text) returns number
end
```

```
function exprUpper(s text) returns text
end
```

```
function exprLower(s text) returns text
end
```

```
function exprTrim(s text) returns text
end
```

```
function exprSplit(s text, delim text) returns list text
end
```

```
function exprJoin(parts list text, delim text) returns text
end
```

```
function exprReplace(s text, from text, to text) returns text
end
end
```

## English

### 1. Overview

AFRILANG library 'std/c/parseexpr' (tier complex, category complex). Exports 7 function(s): `exprLen`, `exprUpper`, `exprLower`, `exprTrim`, `exprSplit`, `exprJoin`, `exprReplace`.

```
'import "std/c/parseexpr"' · 'use parseexpr'
```

```
- `exprLen(s text) → number`
- `exprUpper(s text) → text`
- `exprLower(s text) → text`
- `exprTrim(s text) → text`
- `exprSplit(s text, delim text) → list text`
- `exprJoin(parts list text, delim text) → text`
- `exprReplace(s text, from text, to text) → text`
```

### 2. Installation & import

Add the module to your program:

```
import "std/c/parseexpr"
use parseexpr
```

Tier: complex · Category: Complex modules (c/)  
Advanced domains – graphs, NLP, complex networks.

### 3. API reference

- `exprLen(s text) → number`•  
`exprUpper(s text) → text`•  
`exprLower(s text) → text`•  
`exprTrim(s text) → text`•  
`exprSplit(s text, delim text) → list`•  
`exprJoin(parts list text, delim text) → text`•  
`exprReplace(s text, from text, to text) → text`

### 4. Usage examples

```
import "std/c/parseexpr"
use parseexpr
```

```
create result = exprLen(/* args */)
say result
```

```
create result = exprUpper(/* args */)
say result
```

```
create result = exprLower(/* args */)
say result
```

```
create result = exprTrim(/* args */)
say result
```

```
create result = exprSplit(/* args */)
say result
```

```
create result = exprJoin(/* args */)
say result
```

## 5. Best practices

- Prefer ``import "std/c/parseexpr"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

## 6. Appendix — source

module parseexpr

```
function exprLen(s text) returns number
end
```

```
function exprUpper(s text) returns text
end
```

```
function exprLower(s text) returns text
end
```

```
function exprTrim(s text) returns text
end
```

```
function exprSplit(s text, delim text) returns list text
end
```

```
function exprJoin(parts list text, delim text) returns text
end
```

```
function exprReplace(s text, from text, to text) returns text
end
end
```

## Référence rapide / Quick reference

- Import : `import "std/c/parseexpr"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/parseexpr/>

## Source (excerpt)

```
module parseexpr

function exprLen(s text) returns number
end

function exprUpper(s text) returns text
end

function exprLower(s text) returns text
end

function exprTrim(s text) returns text
end

function exprSplit(s text, delim text) returns list text
end

function exprJoin(parts list text, delim text) returns text
end

function exprReplace(s text, from text, to text) returns text
end
end
```